Cel-Shaded Tech Demo

CS4515 3D Computer Graphics and Animation — Final Project Report

Group 40 · Bogdan Mocanu · Mihai Santai

1. Work per group member

Minimal features

Feature	What was built	Bogdan Mocanu	Mihai Santai
Multiple viewpoints	Top (orthographic), orbit and chase cameras, switching between four views	50%	50%
Advanced shading	Blinn-Phong and PBR (Cook-Torrance with GGX)	100%	0%
Material textures	Albedo, roughness, metallic and ambient occlusion maps	50%	50%
Normal mapping	Tangent-space normal map with a TBN matrix	100%	0%
Environment mapping	Skybox and roughness-dependent reflections	50%	50%
Smooth paths	Closed C1 loop of six cubic Bézier curves, a cart riding it, path toggle	100%	0%
Hierarchical transformations	Robot arm	0%	100%

Extra features

Feature	What was built	Bogdan Mocanu	Mihai Santai
Cel shading	Banded diffuse light, shadow tint, hard highlight	0%	100%
Rim light	Lit-side silhouette highlight	50%	50%
Outlines	Sobel edge detection on depth and normals	100%	0%
HDR and tonemapping	Float framebuffer, exposure, Reinhard tonemapping	100%	0%
Bloom (post-processing glow)	Bright pass and separable Gaussian blur	100%	0%

Feature	What was built	Bogdan Mocanu	Mihai Santai
Shadow mapping	Sun shadow map with PCF	0%	100%
Shadows for multiple light sources	Shadow maps for both point lights	0%	100%
Constant speed along a Bézier curve	Arc-length table	0%	100%

---

2. The demo

Our demo is a small cel-shaded diorama. A gold dragon (the model from the course practicals [1]) sits in the middle, next to four spheres that go from rough to glossy, a cube with a painted-metal texture set, a cylinder and a robot arm, while a glowing cart rides a Bézier track around everything. A sun and two coloured point lights (off at the start) light the scene under a cube-map sky. Everything runs in one application, and every feature can be switched on and off in the **Settings** window.

Input	Action
1, 2, 3, 4	Free, top, orbit and chase camera
W A S D, R / F, hold left mouse	Move and look around (free camera)
Drag with left mouse, scroll	Rotate and zoom (orbit camera)

Every frame goes through these passes:

```
1. Shadow maps    depth seen from the sun and from the two point lights
2. Scene pass     colour, normals and depth into a floating-point framebuffer
3. Bloom          bright parts only, blurred at half resolution
4. Final pass     bloom, exposure, outlines, tonemapping and gamma, to the screen
5. Overlays       debug geometry drawn on top
```

---

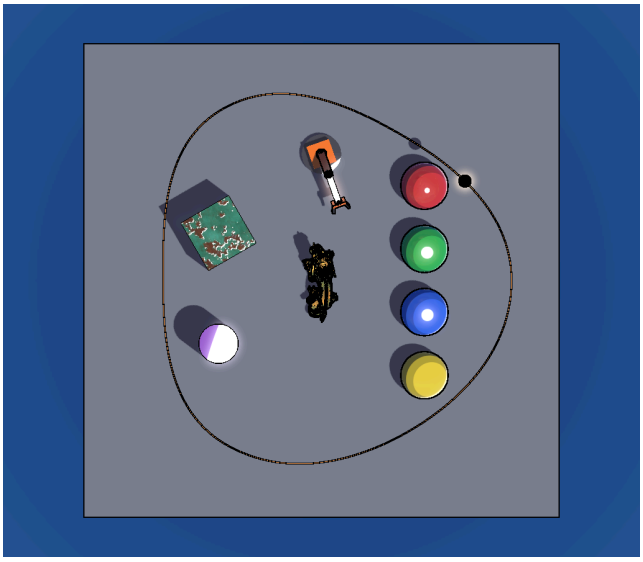
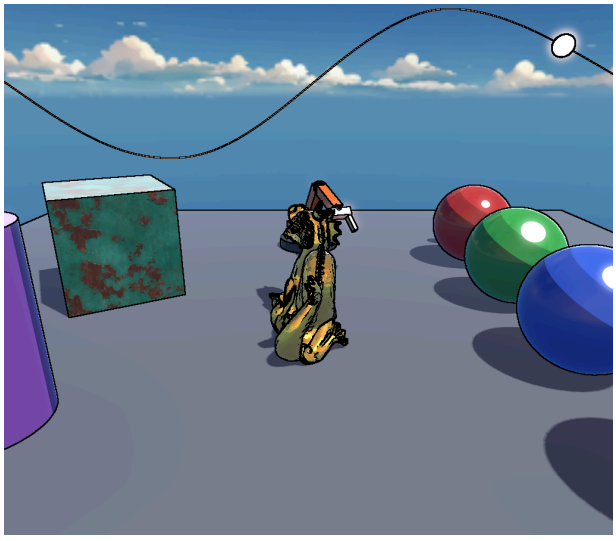
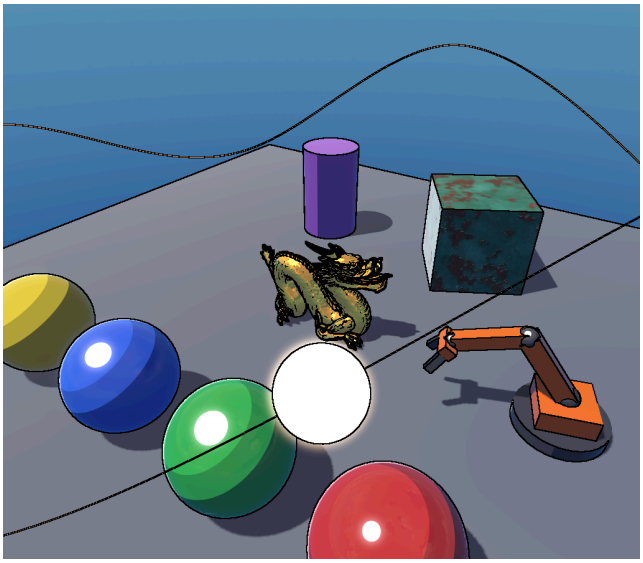
3. Minimal features

3.1 Multiple viewpoints

Bogdan Mocanu and Mihai Santai · UI: Camera → Mode, or keys 1 to 4

There are four cameras: the free camera from Practical 4, an orthographic top view, an orbit camera around the dragon and a chase camera behind the cart. Each camera only produces a view and a projection matrix, built with `glm::lookAt` as in Practical 4 [1], so the rest of the renderer doesn't need to know which one is active. The orbit camera sits on a sphere around the dragon, and dragging and scrolling change its angles and distance. The chase camera sits a little behind and above the cart and keeps looking at the centre of the scene.

Trade-offs. The top view is orthographic so that it reads like a map and shows the track at its true shape, at the cost of losing any sense of depth. The chase camera looks at the scene centre rather than along the track. That loses the ride-along feeling, but the dragon and the other objects stay in frame for the whole lap. The orbit camera's pitch is clamped just below 90°, because at the poles the view direction lines up with the up vector and the image flips.

Free camera	Top camera (orthographic)
	
Orbit camera	**Chase camera**
	

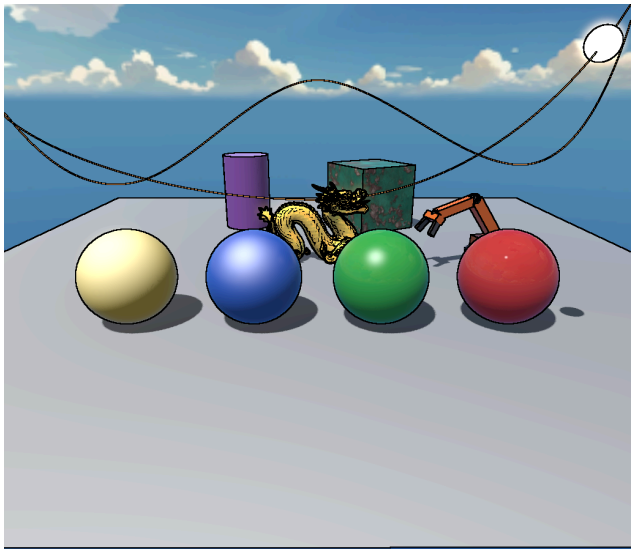
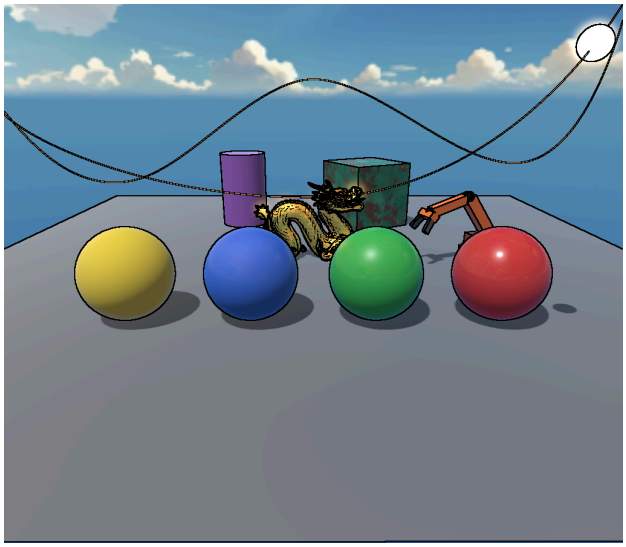
3.2 Advanced shading: Blinn-Phong and PBR

Bogdan Mocanu · UI: Shading → Blinn-Phong / PBR / Cel

The shader has a Blinn-Phong model and a physically based (PBR) model. Both are lit by the sun and two point lights, whose light falls off with the square of the distance. The PBR model is Cook-Torrance [3]: a Lambert diffuse part plus a specular part built from three terms. These are the GGX distribution of microfacet directions [4], Smith's term for microfacets hiding each other [5] and Schlick's Fresnel approximation [6]. Metals get no diffuse light and reflect in their own colour, and non-metals reflect about

4% of the light head-on. Blinn-Phong [7] turns the same roughness value into a shininess exponent, so both models can be compared on the same spheres.

Trade-offs. PBR costs more instructions per light than Blinn-Phong, but it conserves energy, and its parameters (roughness and metallic) have a physical meaning and match the free texture sets we use in Section 3.3. With Blinn-Phong we had to tune the shininess and specular strength by eye, and it easily reflects more light than arrives, which is why the rough ground looks too bright in the left image. We picked GGX over Beckmann because its long tail gives the soft glow around highlights that real materials have, and it is the usual real-time choice [5].

Blinn-Phong	PBR
	

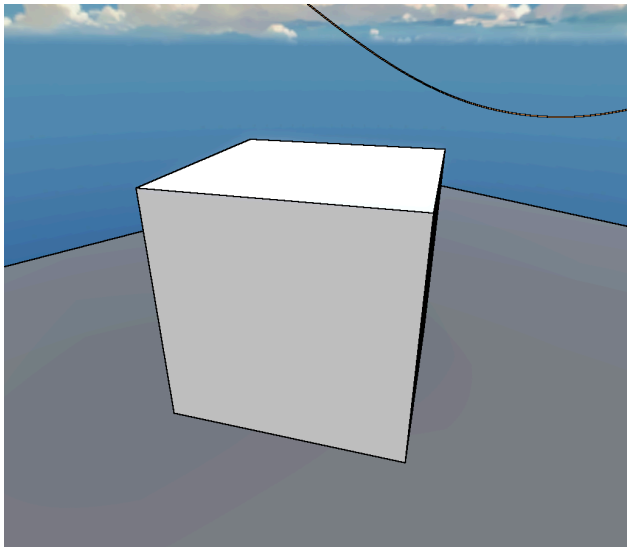
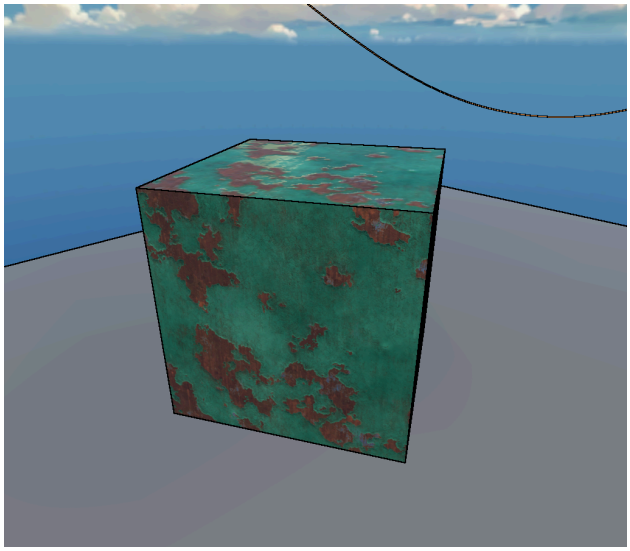
Same scene and lights. The spheres from left to right go from rough (0.90) to glossy (0.15).

3.3 Material textures

Bogdan Mocanu and Mihai Santai · UI: Albedo / Roughness / Metallic / AO maps (top of Settings); constant values per object under Materials

The cube uses the PaintedMetal006 set from ambientCG [17]: albedo, roughness, metalness, ambient occlusion and a normal map. Every map can be switched off on its own. The shader then uses the object's constant value instead, which can also be edited live under Materials. The albedo map is loaded as sRGB so the GPU converts it to linear before lighting, while the other maps are loaded as linear because they store numbers, not colours.

Trade-offs. We chose roughness and metallic maps over separate diffuse, specular and shininess maps because they feed straight into the PBR model. Lighting in linear space and applying gamma once at the end costs nothing per pixel, and it is what makes adding up several lights correct. The price is that every colour in the code must first be converted from sRGB. Keeping a fallback constant for each map adds uniforms and branches to the shader, but it lets us show each map on its own.

All maps off	All maps on
	

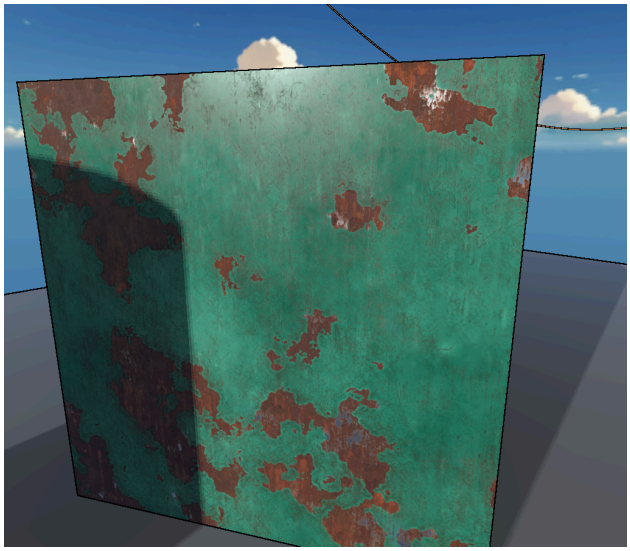
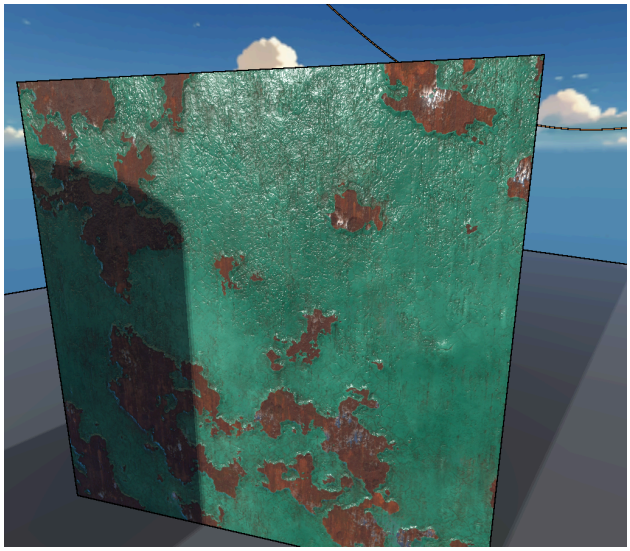
PBR shading. Left: the cube's constant values. Right: all five maps.

3.4 Normal mapping

Bogdan Mocanu · UI: Normal maps, Normal strength

Each vertex stores a tangent next to its normal, so the shader can build a small coordinate frame that follows the surface. The normal map stores a normal per pixel in that frame. The shader rotates it into world space and lights the surface with it instead of the flat mesh normal, which is the idea behind Blinn's bump mapping [8]. A "Normal strength" slider flattens or exaggerates the bumps.

Trade-offs. Tangent-space maps work on any mesh in any orientation, while an object-space map only fits the one mesh it was made for. The cost is an extra vertex attribute and a matrix multiply per pixel. We use the OpenGL version of the ambientCG map, because the DirectX version has its green channel flipped and would light the bumps from the wrong side. The outline pass gets the original mesh normal, so the texture detail doesn't turn into ink lines. Normal mapping only changes the lighting, not the shape, so the cube's silhouette stays perfectly straight.

Normal map off	Normal map on
	

Close-up of the cube with a low sun, so the light grazes the surface and the bumps show.

3.5 Environment mapping

Bogdan Mocanu and Mihai Santai · UI: Environment → Skybox, Reflections

The sky is a cube map drawn around the camera with the camera's position removed, so it always looks infinitely far away. It is drawn last and pushed to the far plane, so it only fills pixels that nothing else covers. Every surface also looks up the cube map in its reflected view direction [9], weighted by the Fresnel term from Section 3.2: metals reflect strongly and in their own colour, other materials mostly at grazing angles. Rougher surfaces read a smaller mipmap of the cube map, so their reflection gets blurrier.

Trade-offs. Drawing the sky last avoids shading sky pixels that objects would cover anyway. The correct way to get rough reflections is to pre-filter the environment with the GGX lobe for each roughness level [5], which needs extra pre-computation passes. Choosing a mipmap by roughness is much simpler and gives the right trend, but the mipmaps are a plain box blur rather than the true GGX blur. The reflections also show only the sky, never the other objects in the scene.

Reflections off	Reflections on
	

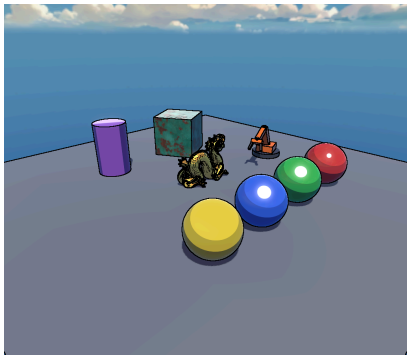
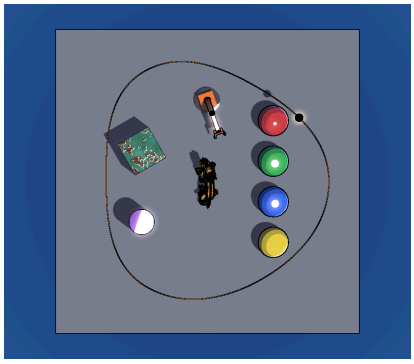
PBR shading. The dragon is metallic, so the sky colour shows up in its reflection.

3.6 Smooth paths: a Bézier loop

Bogdan Mocanu · UI: Bezier path → Show path and cart, Cart speed

The cart's track is a closed loop of six cubic Bézier curves, as taught in the lectures and Exercise 6 [1], defined by six anchor points the track passes through and one handle per anchor. The two inner control points of each curve are the anchor plus or minus its handle. Neighbouring curves share the handle at their joint, so the direction and speed match there: the path is C1-continuous and the cart never jerks. The curve is sampled once at start-up into a line loop, which "Show path and cart" toggles.

Trade-offs. With four free control points per curve, we would have to keep the joints smooth by hand every time a point moved. Shared handles guarantee smoothness and leave fewer numbers to tune. The price is less control: the track can't have a deliberate sharp corner. Moving the curve parameter at a steady rate does not give a steady speed, which Section 4.8 fixes.

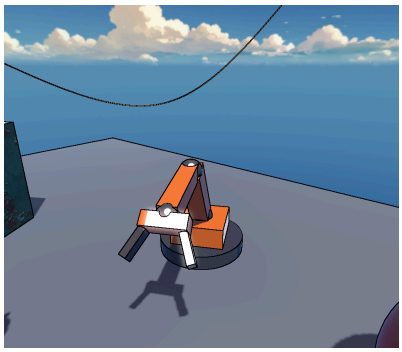
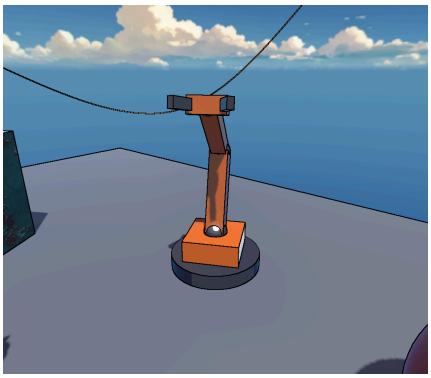
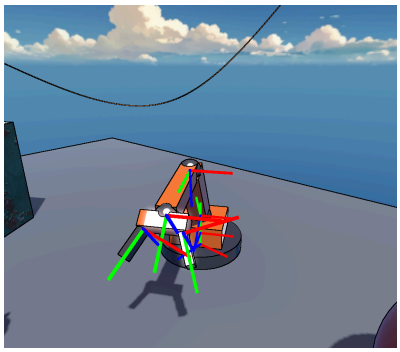
Path hidden	Path shown	Path from the top camera
		

3.7 Hierarchical transformations

Mihai Santai · UI: Robot arm

The robot arm is built in code from the cube, cylinder and sphere meshes, following the robot arm of Practical 1 [1]. It is one chain of ten parts: pedestal, turret, shoulder, upper arm, elbow, forearm, wrist, hand and two fingers. Each joint's frame is its parent's frame, moved to the end of the parent and rotated by the joint's angle, so turning the shoulder carries everything after it. With "Animate" on, the joints follow sine waves of time with a growing delay, so the motion travels down the arm like the snake of Practical 1. With it off, five sliders pose the arm by hand.

Trade-offs. The scale is applied only to the drawn mesh and never stored in a joint frame, otherwise every child would be stretched by its parent and the arm would fall apart. Each part pivots at its joint rather than at its centre, so a rotation hinges it at the end, like a real elbow. Chaining fixed matrices is easy to follow, but without inverse kinematics the arm can't reach for a target and can only play its animation. Driving it from the shared clock keeps the speed independent of the frame rate and lets "Pause" freeze it.

Pose A	Shoulder moved	Joint axes
		

Left and middle: changing only the shoulder angle moves everything above it. Right: the joint axes turn with their joints.

---

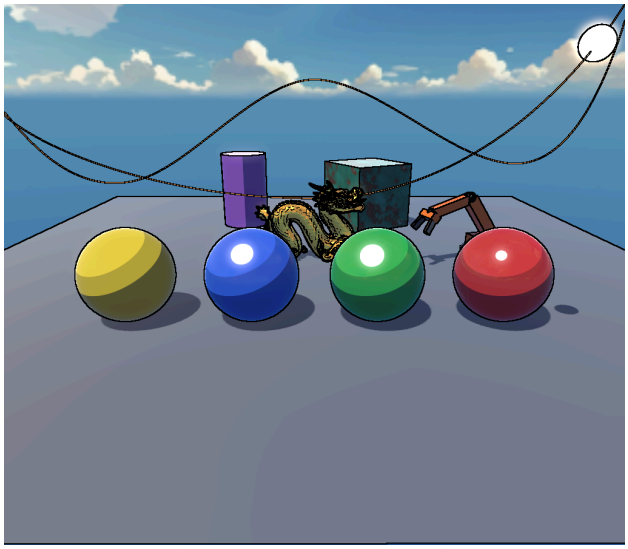
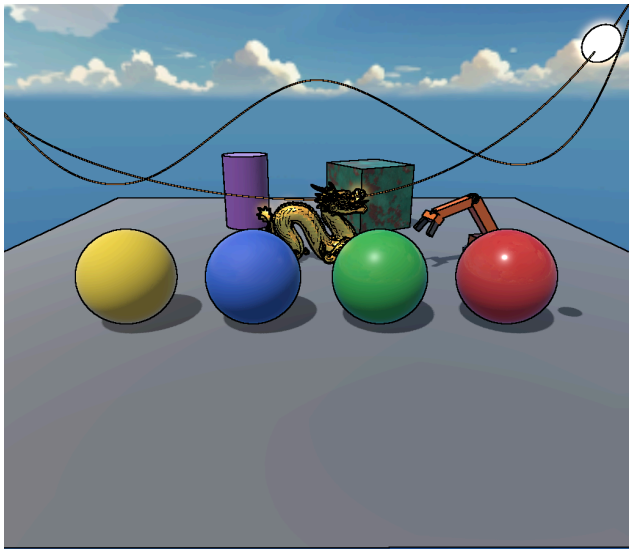
4. Extra features

4.1 Cel shading

Mihai Santai · UI: Shading → Cel, then Bands, Shadow tint, Highlight threshold, Highlight strength

Cel mode turns the smooth diffuse light into a few flat bands, 2 to 5 depending on the "Bands" slider, like a hand-drawn cartoon. Each band edge is softened over about one pixel, so it stays crisp without stair-stepping. A coloured shadow tint, dark blue by default, replaces the grey ambient light, so shadows look cool instead of dirty. The highlight becomes a hard spot where the GGX term passes a threshold, so it still shrinks or disappears as roughness grows.

Trade-offs. This follows the toon shading of Practical 3 [1]. Lake et al. [12] read the bands from a 1D ramp texture, which allows hand-painted gradients. Computing the bands in the shader gives up that freedom, but it needs no texture and lets the band count change live. Reusing GGX for the highlight ties the cartoon look to the same material values as PBR. The price of flat bands is that fine detail, such as the normal-map bumps, partly disappears inside large uniform areas.

Cel	PBR
	

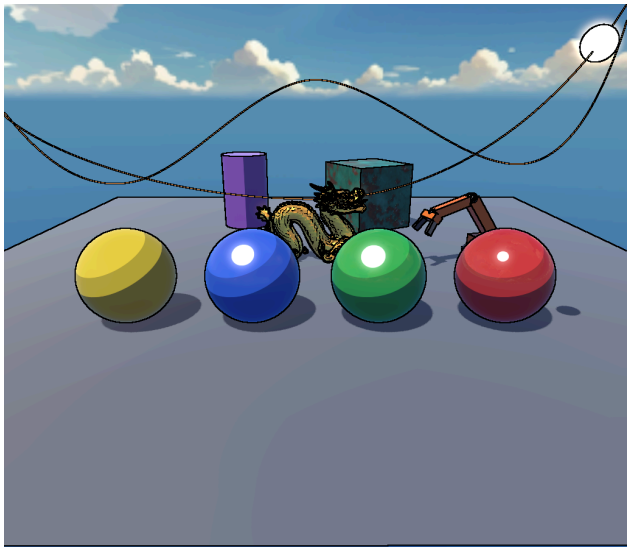
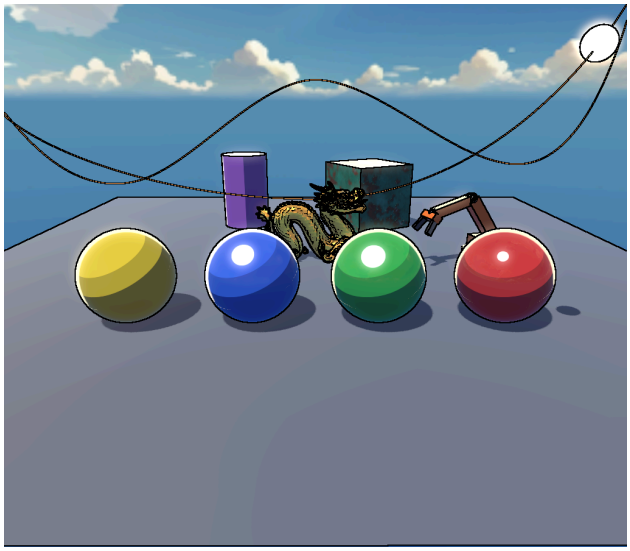
Three bands, and the same scene with smooth PBR shading.

4.2 Rim light

Bogdan Mocanu and Mihai Santai · UI: Shading → Cel → Rim light, Rim color, Rim size; on or off per object under Materials

The rim light adds a thin bright band along an object's silhouette, on the side facing the sun. It is strongest where the surface turns away from the camera, and a mask keeps it to the area the sun reaches. Like the cel bands it ends in a hard edge, and "Rim size" sets its width.

Trade-offs. Tying the rim to the lit side makes it read as backlight from the sun rather than a glow all around the object. The cost is that the shadowed side, where extra separation from the background would help most, gets none. The rim depends on the viewing angle, so a large flat surface lights up across its whole face at grazing angles, which is why the ground has the rim switched off. It costs only a few instructions per pixel.

Rim light off	Rim light on
	

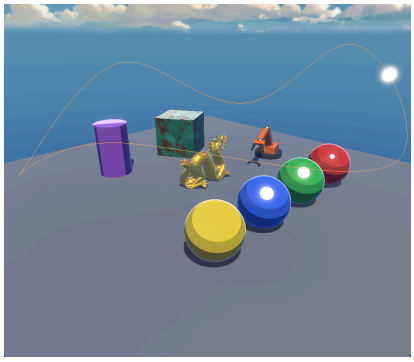
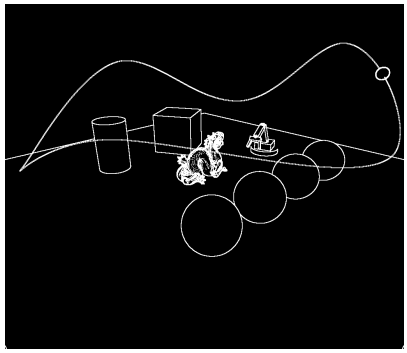
Look at the right edges of the spheres.

4.3 Outlines from edge detection

Bogdan Mocanu · UI: Outlines → Show outlines, Depth threshold, Normal threshold, Thickness, Ink color, Debug: show edges only

During the scene pass we store each pixel's depth and surface normal, and the final pass runs a Sobel edge filter [13] over both. Depth edges catch silhouettes, where one object is in front of another. Normal edges catch creases, where the surface turns sharply, like the cube's edges and the dragon's folds. The depth is first turned back into a real distance and the edge strength is divided by it, so distant objects get outlines too and the floor doesn't turn black at the horizon.

Trade-offs. Working in screen space outlines the whole scene in one pass, however many meshes there are, and finds the inner creases as well, following Saito and Takahashi [11]. The usual alternative, the inverted hull (an enlarged back-face copy of each mesh), only draws silhouettes and needs an extra draw per object, but it allows a different line width per object. Our lines have a fixed pixel width and depend on two thresholds tuned by hand. Even with tuned values, the finely detailed dragon gets very busy with lines.

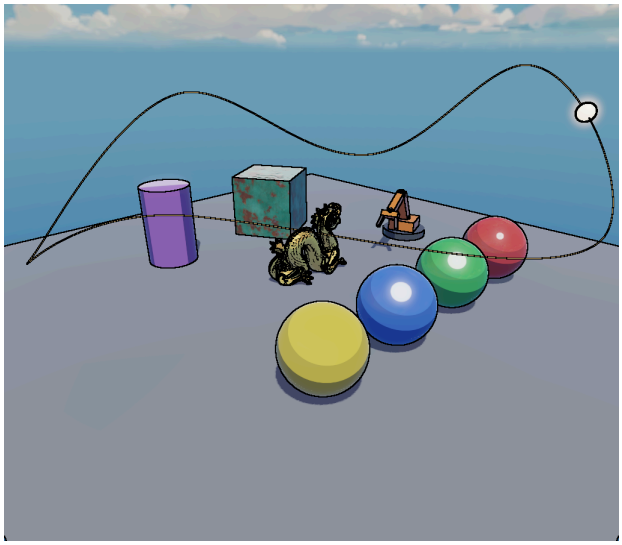
Outlines off	Outlines on	Debug: edges only
		

4.4 HDR rendering and tonemapping

Bogdan Mocanu · UI: HDR → Exposure, Tonemapping (Reinhard)

The scene is rendered into a 16-bit floating-point framebuffer instead of straight to the screen. Colours brighter than white are kept instead of clipped: the sun has intensity 3, and the cart glows at up to 3. The final pass adds the bloom, multiplies by an exposure value and, when enabled, applies Reinhard tonemapping [10], which squeezes any brightness into the range the screen can show. Gamma correction comes last.

Trade-offs. A floating-point buffer uses twice the memory of an 8-bit one, but without it bloom could not tell "bright" from "very bright". Reinhard is the simplest global operator: it never clips, and it leaves dark colours almost untouched. Its downside, visible in the screenshots, is that bright colours lose saturation and the image looks duller, which works against the saturated cel look. A filmic curve would keep more contrast, but it has more parameters to tune.

Exposure 2, no tonemapping	Exposure 2, Reinhard
	

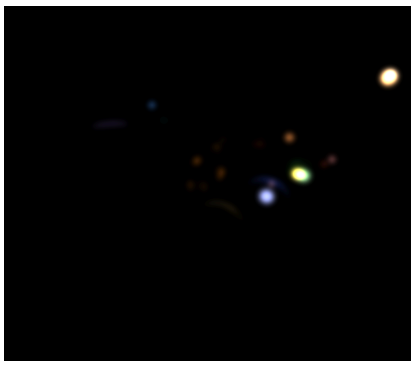
Left: bright areas clip (sky, lit sides of the spheres). Right: Reinhard brings them back into range, at the cost of a duller image.

4.5 Bloom

Bogdan Mocanu · UI: Bloom → Show bloom, Bloom intensity, Bloom threshold, Blur passes, Debug: show bloom only; Emissive colour per object under Materials

Bloom makes very bright parts of the image glow into their surroundings. A bright pass keeps only the part of each pixel that is above a threshold, at half resolution. A Gaussian blur then runs horizontally and vertically, several times, swapping between two textures. The result is added back onto the image before exposure and tonemapping. The cart and the point-light markers glow because their emissive colour is above the threshold.

Trade-offs. Splitting the blur into a horizontal and a vertical pass gives the same result as a full 2D blur, with 18 instead of 81 texture reads per pixel. Working at half resolution makes the glow reach twice as far for a quarter of the cost. Each extra blur pass widens the glow but adds two more full-screen draws. A chain of smaller and smaller mipmaps would give a wider, smoother glow for less work, but it needs a framebuffer per level and more code, so we kept the standard simple approach [2].

Bloom off	Bloom on	Debug: bloom only
		

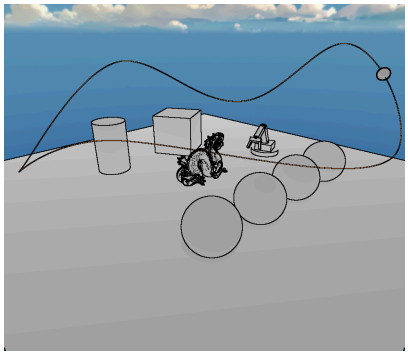
The debug view shows the blurred bright parts that are added to the image.

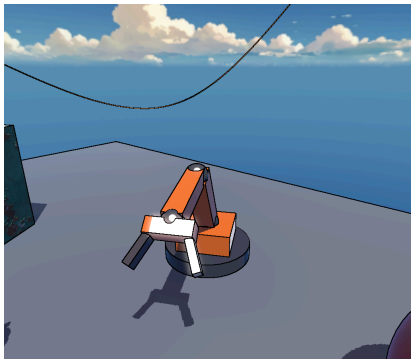
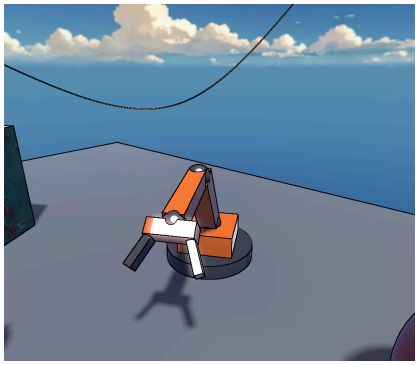
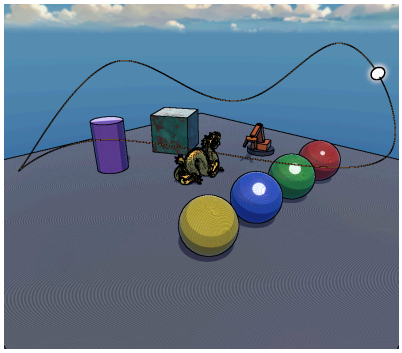
4.6 Shadow mapping

Mihai Santai · UI: Shadows

Before the main pass, the scene's depth is rendered from the sun into a 2048 × 2048 depth texture [14]. The sun's rays are parallel, so this light camera uses an orthographic box around the whole scene, as in Practical 4 [1]. In the main shader, each pixel is projected into the sun's view and compared with the stored depth. If something nearer to the sun is stored there, the pixel is in shadow and loses the sun's light. Percentage-closer filtering [15] averages this test over a small square of neighbouring texels (3 × 3 by default), which softens the shadow's edge.

Trade-offs. A small depth bias stops surfaces from shadowing themselves ("acne", bottom right), but too much bias makes shadows come loose from their objects. A bias of 0.003 removed the acne while the shadows still touched their objects. A fixed box needs no fitting to the camera and suits our small scene, but at some sun angles the far corners of the ground fall outside it and get no shadows. Bigger filter kernels give softer edges but more lookups, 81 per pixel at radius 4. We compare depths by hand instead of using the hardware shadow sampler, which is slower but follows the exercise and keeps every step visible.

Shadows off	Shadows on	Debug: shadow-map depth
		

PCF off, close-up	PCF radius 4, close-up	Bias 0
		

Top right: the depth seen from the sun, darker is nearer. Bottom: the shadow edge is blocky without filtering and soft with radius 4, and with no bias the floor and spheres are covered in acne stripes.

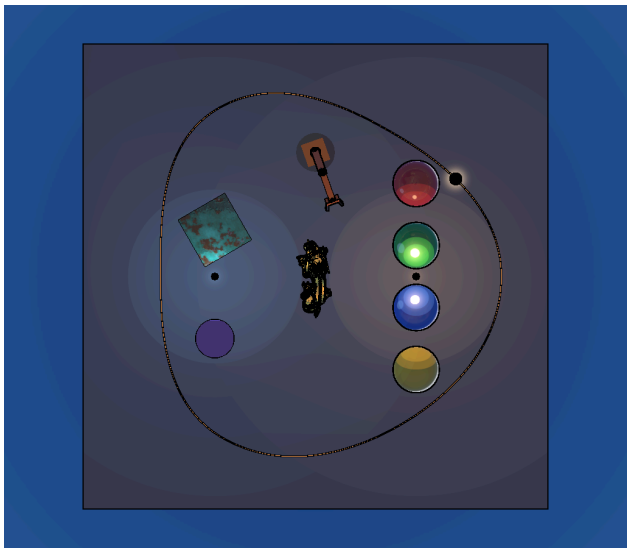
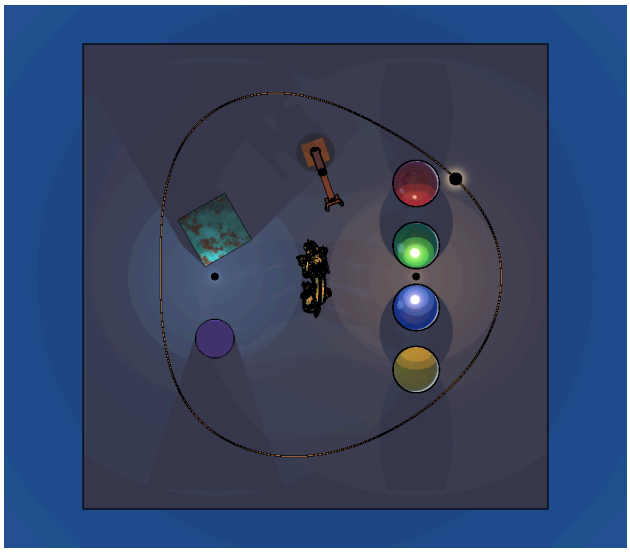
4.7 Shadows for multiple light sources

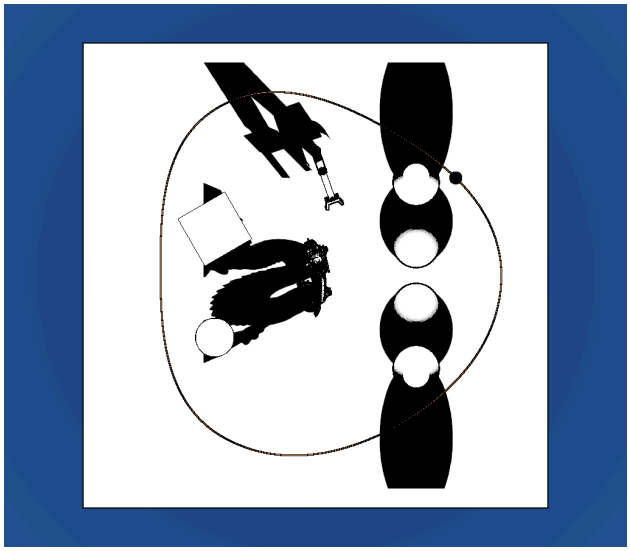
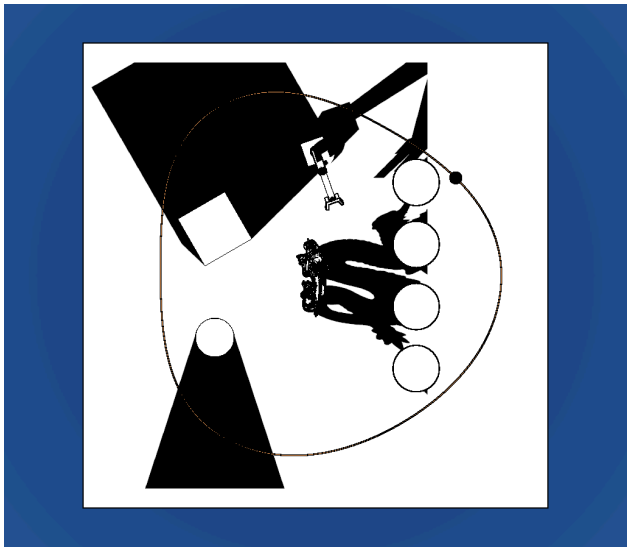
Mihai Santai · UI: Shadows, Point lights

Each of the two point lights has its own 2048 × 2048 shadow map, made by the same code as the sun's. Its light camera sits at the light and looks straight down through a wide 140° perspective cone. The shader runs the same shadow test for all three lights and multiplies each light by its own result, so a point in one light's shadow still receives the other lights. That is why each object casts two coloured shadows in the top view.

Trade-offs. A cube map would cover every direction, but it needs six depth passes per light instead of one. Both lights hang above the scene and everything they light is below them, so one downward cone is enough. The price is that anything more than 70° from straight down, roughly 2.7 units away along the ground, gets no shadow, although the light is already weak there. Perspective depth is less precise far from the light, so the point lights needed their own bias (0.004) to remove ring-shaped acne.

Point-light shadows off	**Point-light shadows on**
	

Off, top view	**On, top view**
	

Debug: orange light	Debug: blue light
	

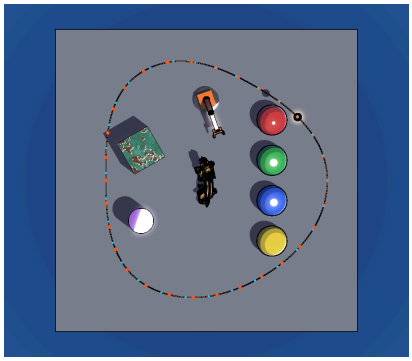
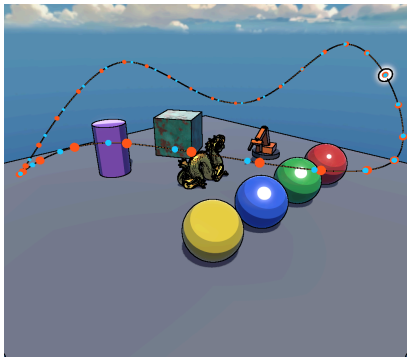
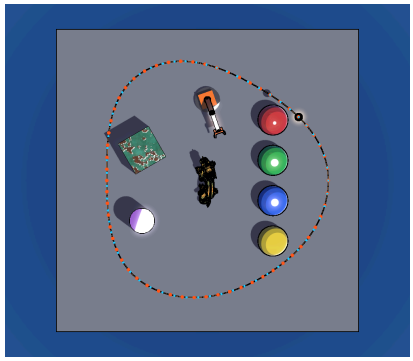
Both point lights on, sun off. In the top view every object has two shadows pointing away from the two lights. In the debug views white is lit and black is in shadow.

4.8 Constant speed along a Bézier curve

Mihai Santai · UI: Constant speed

When the curve parameter grows at a steady rate, the cart still speeds up on wide stretches and slows down in tight turns, because the parameter is not the same as the distance along the curve. So at start-up we sample the track 200 times per curve and store the length covered so far at each sample in a table [16]. Each frame, we work out how far the cart should have travelled, find that distance in the table with a binary search and interpolate the parameter between the two neighbouring samples. Over a lap, the speed now stays between 0.778 and 0.781 units per second, where it used to vary between 0.648 and 0.912.

Trade-offs. The table is built once, so each frame only costs a binary search of about 11 steps. It only needs the path's position function, so the Bézier code didn't change. Measuring the curve with short straight pieces slightly underestimates its length, but at 200 pieces per curve the error is far too small to see. The table would need rebuilding if the track could be edited, and the speed slider keeps its meaning, so a lap takes 20 seconds in both modes.

Markers, top view	Markers, side view	60 markers, top view
		

Orange: equal steps of the parameter. Cyan: equal steps of distance. The orange markers bunch up in the tight turn, while the cyan ones stay evenly spaced.

---

5. References

Course material

[1] CS4515 3D Computer Graphics and Animation, TU Delft, 2026: lecture slides and practical assignments (Practical 1, Practical 3 on shading, Practical 4 on cameras and shadow mapping, Exercise 6 on Bézier curves). The dragon model comes from the Practical 3 material.

Tutorials

[2] J. de Vries. *LearnOpenGL*. https://learnopengl.com. Chapters "Cubemaps", "Normal Mapping", "HDR", "Bloom", "PBR: Theory" and "PBR: Lighting".

Papers

[3] R. L. Cook and K. E. Torrance. A Reflectance Model for Computer Graphics. *ACM Transactions on Graphics* 1(1), 1982.

[4] B. Walter, S. R. Marschner, H. Li and K. E. Torrance. Microfacet Models for Refraction through Rough Surfaces. *Eurographics Symposium on Rendering*, 2007.

[5] B. Karis. Real Shading in Unreal Engine 4. *SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice*, 2013.

[6] C. Schlick. An Inexpensive BRDF Model for Physically-based Rendering. *Computer Graphics Forum* 13(3), 1994.

[7] J. F. Blinn. Models of Light Reflection for Computer Synthesized Pictures. *SIGGRAPH*, 1977.

[8] J. F. Blinn. Simulation of Wrinkled Surfaces. *SIGGRAPH*, 1978.

[9] J. F. Blinn and M. E. Newell. Texture and Reflection in Computer Generated Images. *Communications of the ACM* 19(10), 1976.

[10] E. Reinhard, M. Stark, P. Shirley and J. Ferwerda. Photographic Tone Reproduction for Digital Images. *ACM Transactions on Graphics* 21(3), 2002.

[11] T. Saito and T. Takahashi. Comprehensible Rendering of 3-D Shapes. *SIGGRAPH*, 1990.

[12] A. Lake, C. Marshall, M. Harris and M. Blackstein. Stylized Rendering Techniques for Scalable Real-Time 3D Animation. *NPAR*, 2000.

[13] I. Sobel and G. Feldman. A 3×3 Isotropic Gradient Operator for Image Processing. Talk at the Stanford Artificial Intelligence Project, 1968.

[14] L. Williams. Casting Curved Shadows on Curved Surfaces. *SIGGRAPH*, 1978.

[15] W. T. Reeves, D. H. Salesin and R. L. Cook. Rendering Antialiased Shadows with Depth Maps. *SIGGRAPH*, 1987.

[16] B. Guenter and R. Parent. Computing the Arc Length of Parametric Curves. *IEEE Computer Graphics and Applications* 10(3), 1990.

Assets

[17] ambientCG. *Painted Metal 006* (CC0). https://ambientcg.com/view?id=PaintedMetal006